

Consulta SQL Avançado

Expressões de Tabela Comum (CTE)

Professor

Gustavo Dias
Mestre em Gestão e Desenvolvimento Regional

Consultor em Business Intelligence, com atuação em projetos na área da saúde e varejo.



Contatos:

p.gustavodias@gmail.com

https://www.linkedin.com/in/pgustavodias/

pgustavodias



Introdução

Considere a seguinte situação:

Deseja-se identificar clientes com compras acima de um determinado limite.

Com base no escopo de conhecimento desenvolvido até o momento podemos resolver essa questão através da utilização de subquery conforme o exemplo a seguir:

```sql
-- - CONSULTA 1: CLIENTES COM PEDIDOS APROVADOS CUJA SOMA DO VALOR É MAIOR DO QUE 1000

SELECT
    c.id_cliente,
    c.nome
FROM clientes c
WHERE c.id_cliente IN (
    SELECT p.id_cliente
    FROM pedidos p
    WHERE p.status = 'APROVADO'
    GROUP BY p.id_cliente
    HAVING SUM(p.valor) > 1000
);
```



Introdução

A CTE representa o resultado de consulta nomeada, definida pela cláusula WITH e disponível para a instrução SQL que a sucede.

São especialmente úteis para a organização do raciocínio e para a delimitação e o reaproveitamento do escopo durante a execução da consulta.



Características

Nomeação: são nomeadas e podem ser referenciadas na consulta principal. Isso torna o código mais legível e facilita a manutenção.

Escopo: é visível apenas para a consulta em que foi definida. Ela não está disponível fora dessa consulta.

Recursividade: podem ser recursivas, o que significa que podem chamar a si mesmas dentro da definição. Isso é útil para consultas hierárquicas, como árvores genealógicas ou organogramas.



Características

Reutilização: Você pode referenciar a mesma CTE várias vezes na consulta principal ou em subqueries.

Melhor Leitura do Código: Elas tornam o código mais legível e facilitam a compreensão da lógica da consulta, especialmente em consultas complexas.

Otimização: Os otimizadores de banco de dados podem otimizar consultas que usam CTEs, resultando em um desempenho potencialmente melhor do que consultas equivalentes sem CTEs.



Características

Desempenho: Em alguns casos, uma CTE pode ser mais eficiente do que usar subqueries aninhadas, especialmente quando a mesma subquery é usada várias vezes na consulta.

Modularidade: Elas permitem que você quebre uma consulta complexa em partes menores e mais gerenciáveis, facilitando a manutenção do código.



Características

Facilidade de Depuração: Como as CTEs são nomeadas, você pode incluir comentários explicativos sobre a lógica de cada CTE, facilitando a depuração e o entendimento do código.

Código Mais Limpo: Em muitos casos, o uso de CTEs pode simplificar o código e torná-lo mais limpo e legível.



Aplicação Prática

Considere a seguinte situação:

Deseja-se identificar clientes com compras acima de um determinado limite.

Vejamos a resolução deste mesmo caso agora com a utilização da CTE.

```sql
-- - CONSULTA 2: (CTE) CLIENTES COM PEDIDOS APROVADOS CUJA SOMA DO VALOR É MAIOR DO QUE 1000

WITH total_por_cliente AS (
    SELECT
        id_cliente,
        SUM(valor) AS total_compras
    FROM pedidos
    WHERE status = 'APROVADO'
    GROUP BY id_cliente
)
SELECT
    c.id_cliente,
    c.nome
FROM clientes c
INNER JOIN total_por_cliente t
    ON t.id_cliente = c.id_cliente
WHERE t.total_compras > 1000
```



Escopo, Referência e Múltiplas CTEs

- A CTE existe para a instrução que a define;
- Pode haver mais de uma CTE na mesma cláusula WITH, separadas por vírgula;
- Uma CTE posterior pode referenciar uma CTE definida antes dela;
- Uma CTE pode ser usada na consulta principal sem necessariamente exigir JOIN;
- Ordem importa quando do uso de CTEs encadeadas.

```sql
-- - CONSULTA 3: CLIENTES COM PEDIDOS APROVADOS, VALOR TOTAL E MÉDIO DAS COMPRAS

WITH total_por_cliente AS (
    SELECT
        id_cliente,
        SUM(valor) AS total_compras
    FROM pedidos
    WHERE status = 'APROVADO'
    GROUP BY id_cliente
),
media_compras AS (
    SELECT
        AVG(total_compras) AS media
    FROM total_por_cliente
)
SELECT
    c.nome,
    t.total_compras,
    m.media
FROM total_por_cliente t
INNER JOIN clientes c
    ON c.id_cliente = t.id_cliente
CROSS JOIN media_compras m -- NOVO TIPO DE JOIN
WHERE t.total_compras > m.media
ORDER BY t.total_compras DESC;
```



CTE Melhora Performance?

Não necessariamente.

O plano de execução depende do SGDB e da consulta.

O otimizador pode incorporar a CTE ao plano ou materializar resultados, conforme o produto e o contexto.

É recomendável:

- Comparar planos de execução e medições sem condições equivalentes;
- Como boa prática, priorizar clareza e validar performance.

CTE, Subquery ou tabela temporária?

SUBQUERY

Consulta aninhada em uma expressão ou instrução.

Boa opção para lógica curta e localizada.

CTE

Resultado intermediário nomeado dentro de uma instrução.

Boa opção para etapas lógicas e consultas mais legíveis.

TABELA TEMPORÁRIA

Objeto temporário que pode ser usado em instruções posteriores, conforme o SGBD.

Considere quando precisar reutilizar ou armazenar explicitamente um resultado.



Outras Aplicações

Alguns SGDBs permitem CTEs em instruções como UPDATE, DELETE, INSERTE ou CREATE TABLE.

Não se deve assumir que a mesma sintaxe funcione em todos os bancos ou versões.

Consulte a documentação do SGDB utilizado antes de aplicar o recurso em comandos de alteração.

.prática

Exercícios

Exercício 1 — Clientes com quantidade elevada de pedidos

Contexto: a empresa deseja identificar clientes recorrentes, considerando a quantidade de pedidos aprovados, independentemente do valor financeiro.

Crie uma CTE chamada quantidade_por_cliente que conte quantos pedidos aprovados cada cliente realizou. Na consulta principal, apresente o nome, a quantidade de pedidos e o total financeiro correspondente, considerando apenas clientes com dois ou mais pedidos aprovados.

Exercícios

Exercício 2 — Clientes com compras acima da média geral
Contexto: a gerência deseja identificar os clientes cujo volume de compras aprovadas está acima da média geral dos pedidos aprovados, considerando o valor de cada pedido individualmente.

Crie duas CTEs:
1. pedidos_aprovados: retorna os pedidos aprovados.
2. media_pedidos: calcula o valor médio dos pedidos aprovados.

Na consulta principal, apresente o nome do cliente, o identificador do pedido, o valor do pedido e a média geral, mostrando somente pedidos cujo valor seja superior à média.

Obrigado!



© Todos os direitos reservados ao autor prof. Luiz Gustavo Dias | Conteúdo desenvolvido exclusivamente para aula do curso de Sistemas de Informação da UNIVÁS.